

分治之美

排序与二分 · 7题 · C++ & Python 双语对照

NQ106 快速排序 AcWing 785

给定你一个长度为 n 的整数数列。请你使用快速排序对这个数列按照从小到大进行排序。并将排好序的数列按顺序输出。

输入

输入共两行，第一行包含整数 n 。第二行包含 n 个整数（所有整数均在 $1 \sim 10^9$ 范围内），表示整个数列。（ $1 \leq n \leq 100000$ ）

输出

输出共一行，包含 n 个整数，表示排好序的数列。

来源

AcWing 785

输入

5
3 1 2 4 5

输出

1 2 3 4 5

提示 [原题链接](#)

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

long long *numbers;

void quicksort(long long nums[], int left, int right)
{
    if (left >= right)
        return;
    //选取分割数
    long long target = nums[left]; //选取第一个快排的分割数
    int i = left - 1, j = right + 1;
    while (i < j)
    {
        do
            i++;
        while (nums[i] < target); //i指针定位
        do
            j--;
        while (nums[j] > target); //j指针定位
        if (i < j)
            swap(nums[i], nums[j]); //交换
    }
    //分治
    quicksort(nums, left, j); //j左边的都是不比Target大的
    quicksort(nums, j + 1, right); // j右边的都比Target大
}

int main()
{
    //!cin\cout提速
    ios::sync_with_stdio(false); //来打消iostream的输入输出缓存
    cin.tie(0); //cin.tie(0)来解除cin与cout的绑定,0表示NULL

    int n;
    //创建动态数组
    cin >> n;
    numbers = new long long[n];
    for (int i = 0; i < n; i++)
        cin >> numbers[i];
    quicksort(numbers, 0, n - 1); //注意边界

    for (int i = 0; i < n; i++)
        cout << numbers[i] << " ";

    delete numbers;
    return 0;
}

```

Python

Python solution pending

NQ107 快速选择第k个数 AcWing 786

给定一个长度为n的整数数列，以及一个整数k，请用快速选择算法求出数列的第k小的数是多少。

输入

第一行包含两个整数 n 和 k 。第二行包含 n 个整数（所有整数均在 $1 \sim 10^9$ 范围内），表示整数数列。
数据范围: $1 \leq n \leq 100000, 1 \leq k \leq n$

输出

输出一个整数，表示数列的第 k 小数。

来源

AcWing 786

输入

5 3
2 4 1 5 3

输出

3

提示 [Andy讲解\(2021\) 原题链接](#)

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;

long long *numbers;

long long quickFind(long long nums[], int left, int right, int k)
{
    if (left == right)
        return nums[left]; //只有一个元素, 左开右闭
    //选取分割数
    long long target = nums[left]; //选取第一个快排的分割数
    int i = left - 1, j = right + 1;
    while (i < j)
    {
        do
            i++;
        while (nums[i] < target); //i指针定位
        do
            j--;
        while (nums[j] > target); //j指针定位
        if (i < j)
            swap(nums[i], nums[j]); //交换
    }
    //判断i与k的关系
    int sumofLeft = j - left + 1; //计算左边区间数字个数, 用于与k比较

    if (k <= sumofLeft) //分治
        quickFind(nums, left, j, k); //j左边的都是不比Target大的
    else
        quickFind(nums, j + 1, right, k - sumofLeft); //j右边的都比Target大
}

int main()
{
    //!cin\cout提速
    ios::sync_with_stdio(false); //来打消iostream的输入输出缓存
    cin.tie(0); //cin.tie(0)来解除cin与cout的绑定, 0表示NULL

    int n, k;
    //创建动态数组
    cin >> n >> k;
    numbers = new long long[n];
    for (int i = 0; i < n; i++)
        cin >> numbers[i];
    //查找第k小数, 输出之
    cout << quickFind(numbers, 0, n - 1, k) << endl; //注意边界

    delete numbers;
    return 0;
}

```

Python

Python solution pending

给定你一个长度为 n 的整数数列。请你使用归并排序对这个数列按照从小到大进行排序。并将排好序的数列按顺序输出。

输入

输入共两行，第一行包含整数 n 。第二行包含 n 个整数（所有整数均在 $1 \sim 10^9$ 范围内），表示整个数列。数据范围： $1 \leq n \leq 100000$

输出

输出共一行，包含 n 个整数，表示排好序的数列。

来源

AcWing 787

输入

5
3 1 2 4 5

输出

1 2 3 4 5

提示 [Andy讲解\(2021\)](#) [原题链接](#)

C++

```

#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
const int N = 100007;
int n;
int numbers[N], tmp[N];

void mergeSort(int nums[], int left, int right)
{
    if (left >= right) return;

    int mid = left + (right - left) / 2; // 防止溢出

    mergeSort(nums, left, mid), mergeSort(nums, mid + 1, right); // 先调用归并排序
    // 合二为一
    int k = 0, i = left, j = mid + 1; // 左右两部分数组的起点

    while (i <= mid && j <= right) // 扫描还未抵达终点
        if (nums[i] <= nums[j]) // 把小的放到tmp数组里面
            tmp[k++] = nums[i++];
        else tmp[k++] = nums[j++];

    while (i <= mid) tmp[k++] = nums[i++]; // 把左边剩余的部分插入tmp
    while (j <= right) tmp[k++] = nums[j++]; // 把右边剩余的部分插入tmp

    // 把tmp的部分复制回数组
    for (i = left, k = 0; i <= right; i++, k++) nums[i] = tmp[k];
}

int main()
{
    // 数据大, 使用printf和scanf
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &numbers[i]);

    mergeSort(numbers, 0, n - 1);

    for (int i = 0; i < n; i++) printf("%d ", numbers[i]);

    return 0;
}

```

Python

Python solution pending

NQ109 逆序对的数量 AcWing 788

给定一个长度为 n 的整数数列，请你计算数列中的逆序对的数量。逆序对的定义如下：对于数列的第 i 个和第 j 个元素，如果满足 $i < j$ 且 $a[i] > a[j]$ ，则其为一个逆序对；否则不是。数据范围： $1 \leq n \leq 100000$

输入 第一行包含整数 n ，表示数列的长度。第二行包含 n 个整数，表示整个数列。

输出 输出一个整数，表示逆序对的个数。

来源 AcWing 788

输入

输出

6
2 3 4 5 6 1

5

C++

```
#include <iostream>
#include <cmath>
#include <algorithm>
using namespace std;
typedef long long LL;

const int N = 100007;
int n;
int numbers[N], tmp[N];
//最大的数可能会是  $n^2/2$ 
LL mergeSort(int nums[], int left, int right)
{
    if (left >= right) return 0; //如果左指针越过右指针, 则返回

    int mid = left + (right - left) / 2; //防止溢出

    LL result = mergeSort(nums, left, mid) + mergeSort(nums, mid + 1, right); //先调用归并排序
    //合二为一
    int k = 0, i = left, j = mid + 1; //左右两部分数组的起点

    while (i <= mid && j <= right) //扫描还未抵达终点
        if (nums[i] <= nums[j]) //把小的放到tmp数组里面
            tmp[k++] = nums[i++];
        else {
            //计算逆序对的个数 mid - i + 1
            result += mid - i + 1;
            tmp[k++] = nums[j++];
        }

    while (i <= mid) tmp[k++] = nums[i++]; //把左边剩余的部分插入tmp
    while (j <= right) tmp[k++] = nums[j++]; //把右边剩余的部分插入tmp

    //把tmp的部分复制回数组
    for (i = left, k = 0; i <= right; i++, k++) nums[i] = tmp[k];

    return result;
}

int main()
{
    //数据大, 使用printf和scanf
    scanf("%d", &n);
    for (int i = 0; i < n; i++) scanf("%d", &numbers[i]);

    cout << mergeSort(numbers, 0, n - 1) << endl;

    return 0;
}
```

Python

Python solution pending

给定一个按照升序排列的长度为 n 的整数数组，以及 q 个查询。对于每个查询，返回一个元素 k 的起始位置和终止位置（位置从0开始计数）。如果数组中不存在该元素，则返回“-1 -1”。数据范围 $1 \leq n \leq 100000$
 $1 \leq q \leq 10000$ $1 \leq k \leq 10000$

输入

第一行包含整数 n 和 q ，表示数组长度和询问个数。第二行包含 n 个整数（均在1~10000范围内），表示完整数组。接下来 q 行，每行包含一个整数 k ，表示一个询问元素。

输出

共 q 行，每行包含两个整数，表示所求元素的起始位置和终止位置。如果数组中不存在该元素，则返回“-1 -1”。

来源

AcWing 789

输入

```
6 3
1 2 2 3 3 4
3
4
5
```

输出

```
3 4
5 5
-1 -1
```

提示 原题

C++

```

#include <iostream>
#include <algorithm>
using namespace std;
const int N = 100007;

int nums[N];

int main()
{
    // 第一行包含整数n和q，表示数组长度和询问个数。
    int n, q;
    scanf("%d%d", &n, &q);
    // 第二行包含n个整数（均在1~10000范围内），表示完整数组。
    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);
    // 接下来q行，每行包含一个整数k，表示一个询问元素。
    while(q--){
        // 读入要查找的数，并且二分查找其范围
        int k;
        scanf("%d",&k); // 读入k

        int left = 0, right = n-1; // 初始化left和right
        while (left < right) // 模板2
        {
            int mid = left + right >> 1;
            if (nums[mid] >= k) right = mid; // 模板1,
            else left = mid + 1;
        }
        // left就是左端点值
        // 判断是否找得到
        if (nums[left] != k) cout << "-1 -1" << endl; // 找不到x
        else
        {
            cout << left << " ";
            // 继续寻找右边界
            left = 0, right = n-1; // 重新初始化left, right, 进行第二
            次二分搜索
            while (left < right)
            {
                int mid = left + right + 1 >> 1;
                if (nums[mid] <= k) left = mid;
                else right = mid - 1;
            }
            cout << left << endl;
        }
    }

    return 0;
}

// int bsearch_1(int left, int right)
// {
//     while (left < right)
//     {
//         int mid = left + right >> 1;
//         if (check(mid)) right = mid;
//         else left = mid + 1;
//     }
//     return left;
// }

// int bsearch_2(int left, int right)

```

Python

Python solution pending

```
// {
//     while (left < right)
//     {
//         int mid = left + right + 1 >> 1;
//         if (check(mid)) left = mid;
//         else right = mid - 1;
//     }
//     return left;
// }
```

NQ111 数的三次方根 AcWing 790

给定一个浮点数 n ，求它的三次方根。数据范围： $-10000 \leq n \leq 10000$

输入 共一行，包含一个浮点数 n 。

输出 共一行，包含一个浮点数，表示问题的解。注意，结果保留6位小数。

来源 AcWing 790

输入
1000.00

输出
10.000000

提示 [原题链接](#)

C++

```
#include <iostream>
#include <algorithm>
using namespace std;

int main()
{
    double number;
    scanf("%lf", &number);
    double left = -10000, right = 10000;
    while (right - left >= 1e-8)
    {
        double mid = (right + left) / 2; //二分
        if (mid * mid * mid >= number)
            right = mid;
        else
            left = mid;
    }
    //输出left
    printf("%lf", left);
}
```

Python

Python solution pending

NQ112 菱形 AcWing 727

输入一个奇数 n ，输出一个由*构成的 n 阶实心菱形。菱形中心为第 $(n+1)/2$ 行，每行星号数量以中心对称。

输入 一个奇数 n 。

输出

输出一个由*构成的n阶实心菱形。

来源

AcWing 727

输入

5

输出

```

  *
 ***
*****
 ***
  *

```

提示 数据范围: $1 \leq n \leq 99$

C++

```

#include <iostream>
#include <cmath>

using namespace std;

int main()
{
    int n;
    cin >> n;

    int cx = n / 2, cy = n / 2;
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            if (abs(i - cx) + abs(j - cy) <= n / 2) cout <<
                '*';
        else cout << ' ';
        cout << endl;
    }

    return 0;
}

```

Python

Python solution pending